Aprendizaje de Máquina
Sesión 24: Árboles de decisión y ensambles de árboles

Angel Rodríguez

Examen parcial II

- Martes 10 de Noviembre

¿Qué es un árbol de decisión?

- Modelo supervisado que puede resolver problemas de **clasificación** y **regresión**.
- Cada nodo interno contiene una condición, por ejemplo $x_j \leq t$.
- Cada rama representa el resultado de esa condición.
- Cada hoja proporciona una predicción.
- Puede capturar relaciones no lineales sin escalar ni centrar los atributos.

Ejemplo

¿La longitud del pétalo es menor o igual a 2,45 cm? Si sí, predecimos *setosa*; si no, hacemos otra pregunta.

Entrenar un árbol con Iris

Usaremos dos características de las flores y limitaremos la profundidad a dos niveles.

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier

iris = load_iris(as_frame=True)
X = iris.data[["petal length (cm)", "petal width (cm)"]].values
y = iris.target

arbol = DecisionTreeClassifier(
    max_depth=2, random_state=42
)
arbol.fit(X, y)
```

Visualizar el árbol entrenado

```
from sklearn.tree import export_graphviz

export_graphviz(
    arbol,
    out_file="iris_arbol.dot",
    feature_names=["longitud", "anchura"],
    class_names=iris.target_names,
    filled=True, rounded=True
)

from graphviz import Source

Source.from_file("iris_arbol.dot")
```

Observación

Cada nodo muestra una regla, el número de muestras, la distribución de clases y la clase predicha.

Cómo se realiza una predicción

1. Comenzamos en la **raíz**.
2. Evaluamos la condición del nodo.
3. Si se cumple, avanzamos por la rama izquierda; de lo contrario, por la derecha.
4. Repetimos hasta alcanzar una **hoja**.
5. La hoja determina la predicción.

Ejemplo Iris

- Longitud $\leq$ 2,45: *setosa*.
- Longitud $>$ 2,45 y anchura $\leq$ 1,75: *versicolor*.
- Longitud $>$ 2,45 y anchura $>$ 1,75: *virginica*.

Interpretación de los nodos

- **samples**: número de ejemplos de entrenamiento que llegan al nodo.
- **value**: cantidad de ejemplos de cada clase (o cantidades ponderadas).
- **class**: clase mayoritaria, usada para predecir.
- **gini**: medida de mezcla de clases dentro del nodo.

Pregunta

Si una hoja contiene 49 flores versicolor y 5 virginica, ¿qué clase predice? ¿Con qué confianza empírica?

Impureza de Gini

Para un nodo i con K clases, definimos

$$\boxed{G_i = 1 - \sum_{k=1}^{K} p_{i,k}^2}$$

donde $p_{i,k}$ es la proporción de ejemplos de clase k en el nodo i.

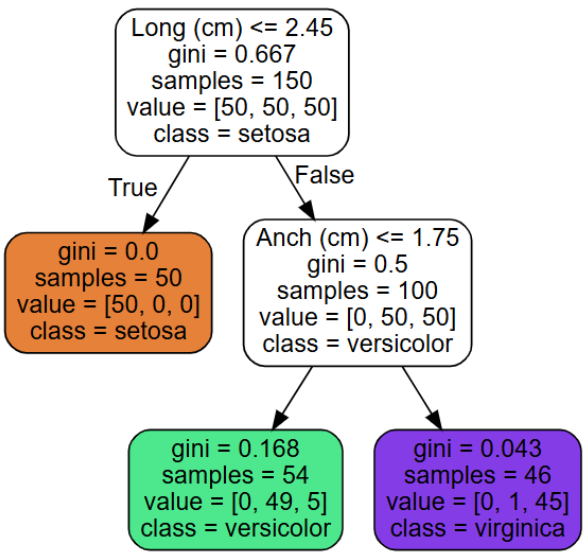
- Si todas las observaciones pertenecen a una clase, $G_i = 0$.
- Cuanto más mezcladas estén las clases, mayor es la impureza.

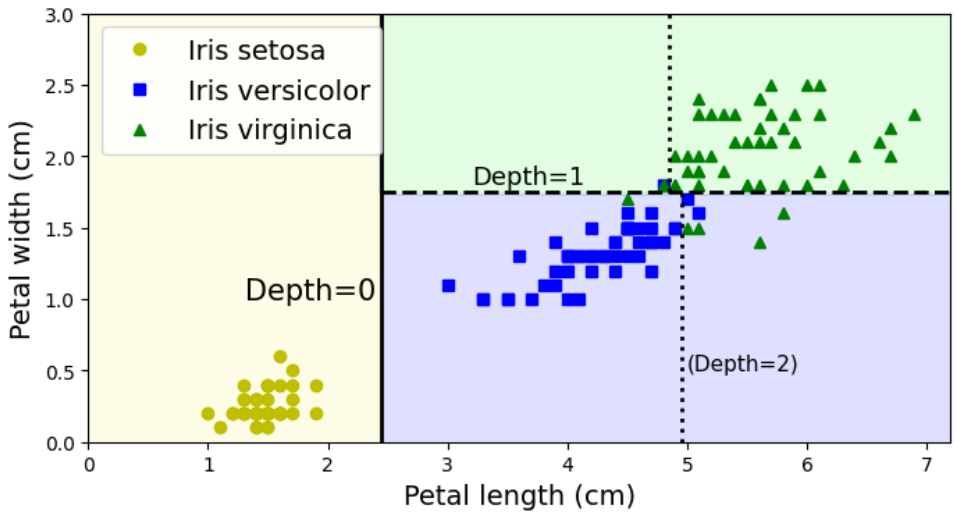
Ejemplo

Para proporciones $(0, 49/54, 5/54)$:

$$G = 1 - (49/54)^2 - (5/54)^2 \approx 0{,}168.$$

Ejemplo


Long (cm) <= 2.45
gini = 0.667
samples = 150
value = [50, 50, 50]
class = setosa
True
False
gini = 0.0
samples = 50
value = [50, 0, 0]
class = setosa
Anch (cm) <= 1.75
gini = 0.5
samples = 100
value = [0, 50, 50]
class = versicolor
gini = 0.168
samples = 54
value = [0, 49, 5]
class = versicolor
gini = 0.043
samples = 46
value = [0, 1, 45]
class = virginica


Iris setosa
Iris versicolor
Iris virginica
Depth=1
Depth=0
(Depth=2)
Petal width (cm)
Petal length (cm)
0.0
0.5
1.0
1.5
2.0
2.5
3.0
0
1
2
3
4
5
6
7

Probabilidades estimadas en las hojas

El árbol estima la probabilidad de clase usando la proporción de ejemplos de esa clase en la hoja:

$$\hat{P}(Y = k \mid X = x) = \frac{m_{hoja,k}}{m_{hoja}},$$

donde:

- m_{hoja}: número total de muestras que llegaron a esa hoja durante el entrenamiento.
- $m_{hoja,k}$: número de esas muestras que pertenecen a la clase k.
- k: clase para la cual queremos estimar la probabilidad.

```
print(arbol.predict_proba([[5, 1.5]]).round(3))
print(arbol.predict([[5, 1.5]]))
```

Resultado esperado: probabilidades $(0, 0{,}907, 0{,}093)$ y clase 1.

Limitación Todos los puntos que llegan a la misma hoja reciben exactamente las mismas probabilidades.

Algoritmo CART: construcción del árbol

CART construye el árbol mediante un procedimiento **recursivo y greedy**.

1. Comienza con todos los ejemplos de entrenamiento en el nodo raíz.
2. Para cada característica k, considera diferentes umbrales t_k.
3. Evalúa la impureza ponderada de los nodos que produciría cada división:

$$J(k, t_k) = \frac{m_{izq}}{m} G_{izq} + \frac{m_{der}}{m} G_{der},$$

   y selecciona la característica y el umbral que minimizan la función de costo:

$$(k^*, t^*) = \arg \min_{k, t_k} J(k, t_k).$$

4. Divide los ejemplos en dos nodos hijos y repite el procedimiento en cada uno.
5. **Criterios de parada:** nodo puro, profundidad máxima, número mínimo de muestras, etc.

CART: búsqueda de la mejor división

Para cada característica numérica, CART considera umbrales entre valores consecutivos ordenados.

Ejemplo: una característica representa las horas de estudio de seis estudiantes.

$$x_k = [1, 2, 3, 4, 5, 6]$$

Los umbrales candidatos son: $t_k \in \{1{,}5,\ 2{,}5,\ 3{,}5,\ 4{,}5,\ 5{,}5\}$.
Supongamos que las clases son: $y = [0, 0, 0, 1, 1, 1]$.

Umbral	**Distribución izquierda/derecha**	$J(k, t_k)$
1,5	$(1, 0)/(2, 3)$	0,400
2,5	$(2, 0)/(1, 3)$	0,250
3,5	$(3, 0)/(0, 3)$	0,000
4,5	$(3, 1)/(0, 2)$	0,250
5,5	$(3, 2)/(0, 1)$	0,400

Selección

CART elige $t_k^* = 3{,}5$, porque produce dos nodos puros y minimiza $J(k, t_k)$.

Complejidad computacional

Sea m el número de ejemplos y n el número de características.

- En un árbol aproximadamente balanceado, la profundidad es del orden de $\log_2 m$.
- **Predicción**: aproximadamente $O(\log m)$ comparaciones por instancia.
- **Entrenamiento**: aproximadamente $O(nm \log m)$ (por nodo) con búsqueda exhaustiva de las divisiones candidatas en el nodo.
- Limitar profundidad y atributos considerados puede acelerar el entrenamiento.

Entropía como criterio de impureza

Además de Gini, podemos utilizar la **entropía**:

$$H_i = -\sum_{k:p_{i,k}>0} p_{i,k} \log_2 p_{i,k}$$

- $H_i = 0$ cuando el nodo contiene una sola clase.
- Una mezcla equilibrada de clases aumenta la entropía.
- Gini y entropía suelen generar árboles similares.
- Gini suele ser algo más rápido de calcular.
- Cuando difieren, entropía genera árboles más balanceados.

Ejemplo

Para $(49/54, 5/54)$, $H \approx 0{,}445$ bits.

Comparar Gini y entropía en Python

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier

iris = load_iris(as_frame=True)
X = iris.data[["petal length (cm)", "petal width (cm)"]].values
y = iris.target

arbol_gini = DecisionTreeClassifier(
    max_depth=3, criterion="gini", random_state=42
)
arbol_ent = DecisionTreeClassifier(
    max_depth=3, criterion="entropy", random_state=42
)

arbol_gini.fit(X, y)
arbol_ent.fit(X, y)

print(arbol_gini.get_depth(), arbol_gini.get_n_leaves())
print(arbol_ent.get_depth(), arbol_ent.get_n_leaves())
```

Visualizar los árboles:

```
from sklearn.tree import export_graphviz
from graphviz import Source

export_graphviz(
    arbol_gini,
    out_file="arbol_gini.dot",
    feature_names=["longitud", "anchura"],
    class_names=iris.target_names,
    filled=True, rounded=True
)
Source.from_file("arbol_gini.dot")
```

```
export_graphviz(
    arbol_ent,
    out_file="arbol_ent.dot",
    feature_names=["longitud", "anchura"],
    class_names=iris.target_names,
    filled=True, rounded=True
)
Source.from_file("arbol_ent.dot")
```

Visualizar fronteras de decisión

```python
import numpy as np
import matplotlib.pyplot as plt

def plot_decision_boundary(clf, X, y):
    x1, x2 = np.meshgrid(
        np.linspace(X[:, 0].min() - 0.5, X[:, 0].max() + 0.5, 100),
        np.linspace(X[:, 1].min() - 0.5, X[:, 1].max() + 0.5, 100)
    )
    X_new = np.c_[x1.ravel(), x2.ravel()]
    y_pred = clf.predict(X_new).reshape(x1.shape)

    plt.contourf(x1, x2, y_pred, alpha=0.3, cmap="Pastel1")
    plt.contour(x1, x2, y_pred, levels=[0.5, 1.5], colors="black")

    for idx in range(max(y) + 1):
        plt.scatter(X[y == idx, 0], X[y == idx, 1], label=f"Clase {idx}")

    plt.xlabel(r"$x_1$")
    plt.ylabel(r"$x_2$")
    plt.legend()
    plt.show()
```

Visualizar árboles

```
plot_decision_boundary(arbol_gini, X, y)

plot_decision_boundary(arbol_ent, X, y)
```

Sobreajuste y regularización

Un árbol sin restricciones puede adaptarse demasiado a los ejemplos de entrenamiento.

- **Árbol profundo**: mayor flexibilidad, pero mayor riesgo de sobreajuste.
- **Árbol poco profundo**: menos varianza, pero puede subajustar.
- Regularizar significa limitar su capacidad de ajustar detalles del conjunto de entrenamiento.

Hiperparámetros de regularización

Hiperparámetro	Efecto
`max_depth`	Limita la profundidad máxima.
`min_samples_split`	Mínimo de muestras para dividir un nodo.
`min_samples_leaf`	Mínimo de muestras en cada hoja.
`max_leaf_nodes`	Limita el número de hojas.
`max_features`	Limita atributos evaluados en cada división.
`min_impurity_decrease`	Exige una mejora mínima de impureza.
`ccp_alpha`	Controla la *poda de mínima complejidad-costo* o MCCP (minimal cost-complexity pruning). Mayor `ccp_alpha`, más poda. Default = 0.

Regla práctica

Aumentar restricciones reduce la complejidad del árbol; demasiada regularización puede causar subajuste.
Reducir `max_depth` y aumentar `min_samples_leaf` suelen ser buenos puntos de partida.

Poda de árboles

Otra forma de regularización es **podar** ramas que aportan poca mejora.

- Se puede crecer un árbol y después eliminar subárboles innecesarios.
- En scikit-learn, `ccp_alpha` controla la poda por complejidad de costo.
- Un valor mayor suele producir un árbol más pequeño.

Poda por complejidad-costo (MCCP)

CART puede construir árboles muy profundos que **sobreajustan** los datos de entrenamiento.

Minimal Cost-Complexity Pruning (MCCP) simplifica el árbol eliminando ramas que aportan poca mejora respecto a su complejidad.

Función objetivo (penaliza el tamaño de árbol)

$$\boxed{R_\alpha(T) = R(T) + \alpha|T|}$$

- $R(T) = \sum_{\ell \in \text{hojas}(T)} \frac{m_\ell}{m} G_\ell$: impureza total ponderada de las hojas.
- $|T|$: número de hojas del árbol.
- $\alpha \geq 0$: penalización por complejidad.

La idea es encontrar un equilibrio entre dos objetivos:

1. Tener poca impureza: $R(T)$.
2. Tener un árbol pequeño: $\alpha|T|$.

MCCP: ¿qué ramas se eliminan?

MCCP utiliza una estrategia llamada **weakest-link pruning** (poda del eslabón más débil).

1. Se construye inicialmente un árbol grande.
2. Para cada nodo interno, se evalúa el costo de sustituir todo su subárbol por una hoja.
3. Se calcula:

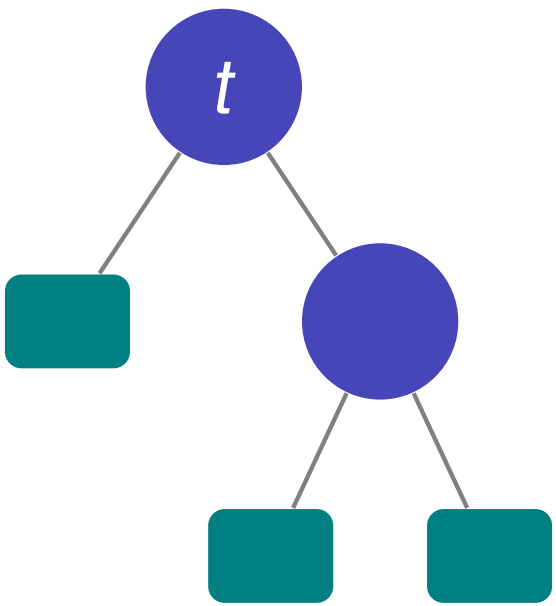
$$\alpha_{eff}(t) = \frac{R(t) - R(T_t)}{|T_t| - 1}$$

4. Se poda el subárbol con menor α_{eff} y con $\alpha_{eff} \leq$ `cpp_alpha`.
5. Se repite el procedimiento para generar árboles progresivamente más pequeños.

¿Qué significa α_{eff}?

Supongamos que tenemos un nodo interno t y el subárbol T_t que comienza en ese nodo:

Antes de podar



3 hojas

$R_\alpha(T_t) = ??$

Después de podar



1 hoja

$R_\alpha(t) = ??$

¿Qué sucede cuando podamos una rama?

Consideremos un nodo interno t y el subárbol T_t que comienza en él. Tenemos dos opciones:

- **Conservar el subárbol T_t:**

$$R_\alpha(T_t) = R(T_t) + \alpha|T_t|$$

- **Podar el subárbol y sustituirlo por una hoja t:**

$$R_\alpha(t) = R(t) + \alpha$$

Al podar:

- La impureza generalmente aumenta.
- El número de hojas disminuye.
- La penalización por complejidad disminuye.

Objetivo

Determinar cuándo la reducción de complejidad compensa el aumento de impureza.

Deducción de α_{eff}

Buscamos el valor de α para el cual conservar y podar el subárbol tienen el mismo costo:

$$R_\alpha(T_t) = R_\alpha(t)$$

Sustituyendo:

$$R(T_t) + \alpha|T_t| = R(t) + \alpha$$

Y haciendo cuentas, obtenemos el:

Alpha efectivo

$$\boxed{\alpha_{eff}(t) = \frac{R(t) - R(T_t)}{|T_t| - 1}}$$

Interpretación de α_{eff}

$$\boxed{\alpha_{eff}(t) = \frac{R(t) - R(T_t)}{|T_t| - 1}}$$

- **Numerador:** $R(t) - R(T_t)$

  Mide cuánto aumenta la impureza al sustituir el subárbol por una hoja.

- **Denominador:** $|T_t| - 1$

  Representa el número neto de hojas eliminadas.

Interpretación

α_{eff} mide el **aumento de impureza por cada hoja eliminada**.

- α_{eff} pequeño: podemos simplificar el árbol perdiendo poca calidad de ajuste.
- α_{eff} grande: la rama aporta una reducción importante de impureza en relación con su complejidad.

Ejemplo: regularización con make_moons

```
from sklearn.datasets import make_moons
from sklearn.tree import DecisionTreeClassifier

X_m, y_m = make_moons(n_samples=150, noise=0.2, random_state=42)

sin_regularizar = DecisionTreeClassifier(random_state=42)
sin_regularizar_ent = DecisionTreeClassifier(random_state=42,criterion="
    entropy")
regularizado = DecisionTreeClassifier(min_samples_leaf=5, random_state
    =42)


sin_regularizar.fit(X_m, y_m)
regularizado.fit(X_m, y_m)
sin_regularizar_ent.fit(X_m, y_m)
```

Visualizar ejemplo *moons*

```
plot_decision_boundary(sin_regularizar, X_m, y_m)
```

```
plot_decision_boundary(sin_regularizar_ent, X_m, y_m)
```

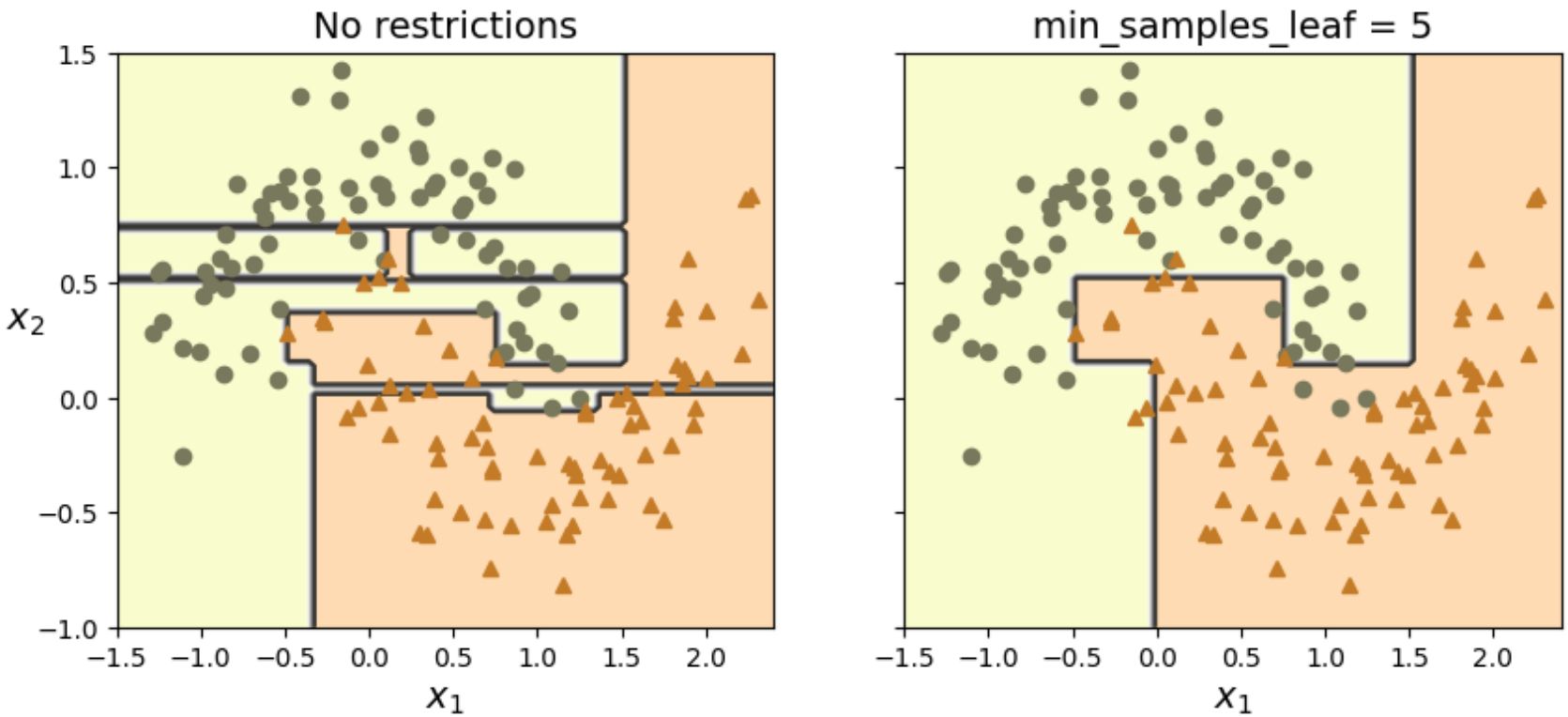
```
plot_decision_boundary(regularizado, X_m, y_m)
```

Visualizar entropía vs gini:

```
export_graphviz(
    sin_regularizar,
    out_file="sin_regularizar.dot",
    feature_names=["longitud", "anchura"],
    class_names=iris.target_names,
    filled=True, rounded=True
)
Source.from_file("sin_regularizar.dot")
```

```
export_graphviz(
    sin_regularizar_ent,
    out_file="sin_regularizar_ent.dot",
    feature_names=["longitud", "anchura"],
    class_names=iris.target_names,
    filled=True, rounded=True
)
Source.from_file("sin_regularizar_ent.dot")
```

Ejemplo:



Árboles de regresión

En regresión, las hojas predicen **valores numéricos** en lugar de clases.

- Cada hoja predice la media de los valores objetivo de los ejemplos que llegan a ella.
- El árbol divide el espacio en regiones con predicciones constantes.
- CART selecciona divisiones que reducen el error cuadrático medio (MSE).

Comparación

Clasificación: pureza de clases. Regresión: dispersión de los valores objetivo.

Entrenar un árbol de regresión

```
import numpy as np
from sklearn.tree import DecisionTreeRegressor
from sklearn.tree import export_graphviz
from graphviz import Source

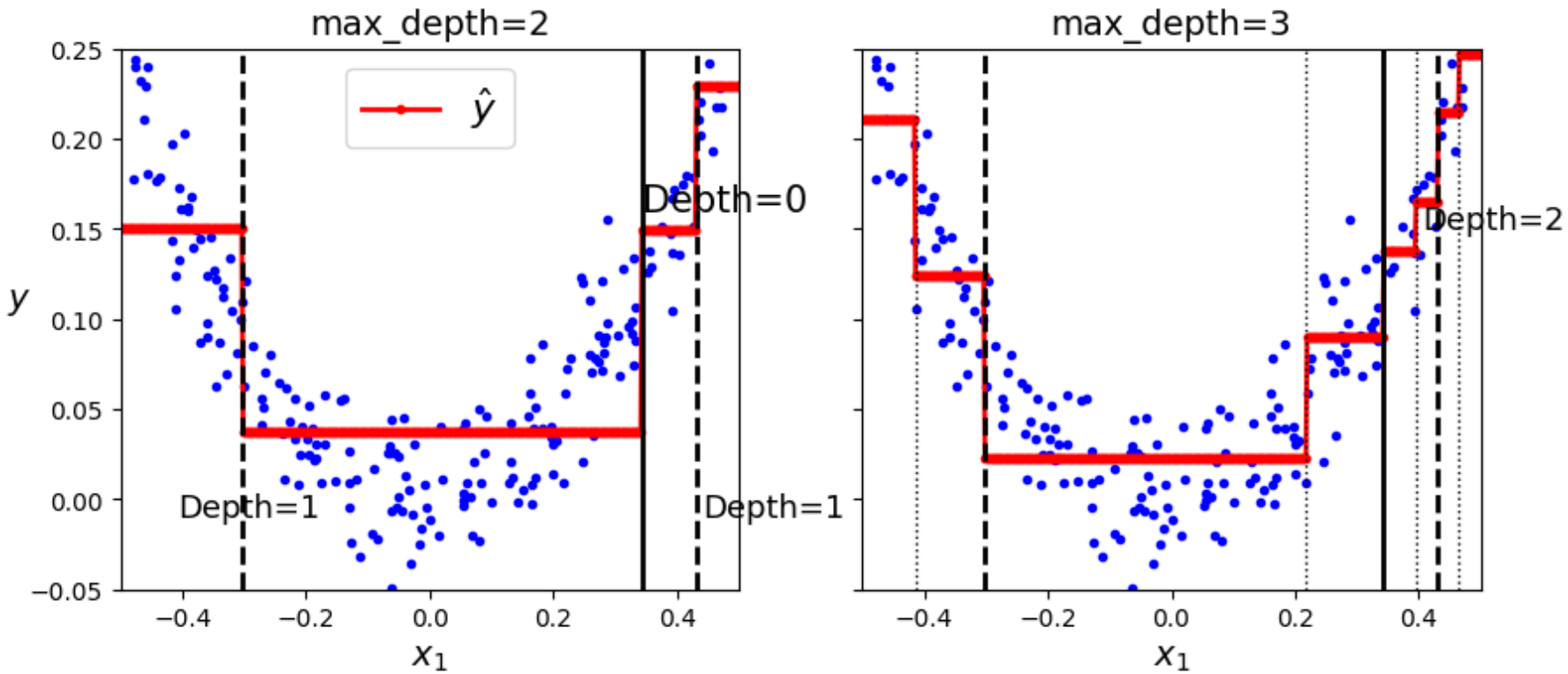
rng = np.random.default_rng(42)
X_q = rng.random((200, 1)) - 0.5
y_q = X_q[:, 0]**2 + 0.025 * rng.standard_normal(200)

reg = DecisionTreeRegressor(max_depth=2, random_state=42)
reg.fit(X_q, y_q)

export_graphviz(
    reg,
    out_file="arbol_reg.dot",
    feature_names=["x1"],
    rounded=True,
    filled=True
)
Source.from_file("arbol_reg.dot")
```

Observación:

Aunque la relación verdadera es cuadrática, el árbol aproxima la curva mediante escalones:



CART para regresión

El costo de una división se calcula como

$$\boxed{J(k, t_k) = \frac{m_{izq}}{m} MSE_{izq} + \frac{m_{der}}{m} MSE_{der}}$$

con

$$MSE_{nodo} = \frac{1}{m_{nodo}} \sum_{i \in nodo} (y_i - \bar{y}_{nodo})^2,$$

$$\bar{y}_{nodo} = \frac{\sum_{i \in nodo} y_i}{m_{nodo}},$$

Interpretación

Se buscan regiones cuyos valores objetivo estén lo más cerca posible de su media.

Regularizar un árbol de regresión

```
import matplotlib.pyplot as plt

reg_libre = DecisionTreeRegressor(random_state=42)
reg_suave = DecisionTreeRegressor(min_samples_leaf=10, random_state=42)

reg_libre.fit(X_q, y_q)
reg_suave.fit(X_q, y_q)

x_grid = np.linspace(-0.5, 0.5, 400).reshape(-1, 1)
plt.scatter(X_q[:, 0], y_q, s=12, alpha=0.4)
plt.plot(x_grid, reg_libre.predict(x_grid), label="Sin regularizar")
plt.legend()
plt.show()
```

```
plt.scatter(X_q[:, 0], y_q, s=12, alpha=0.4)
plt.plot(x_grid, reg_suave.predict(x_grid), label="Regularizado")
plt.legend()
plt.show()
```

Sensibilidad a la orientación de los ejes

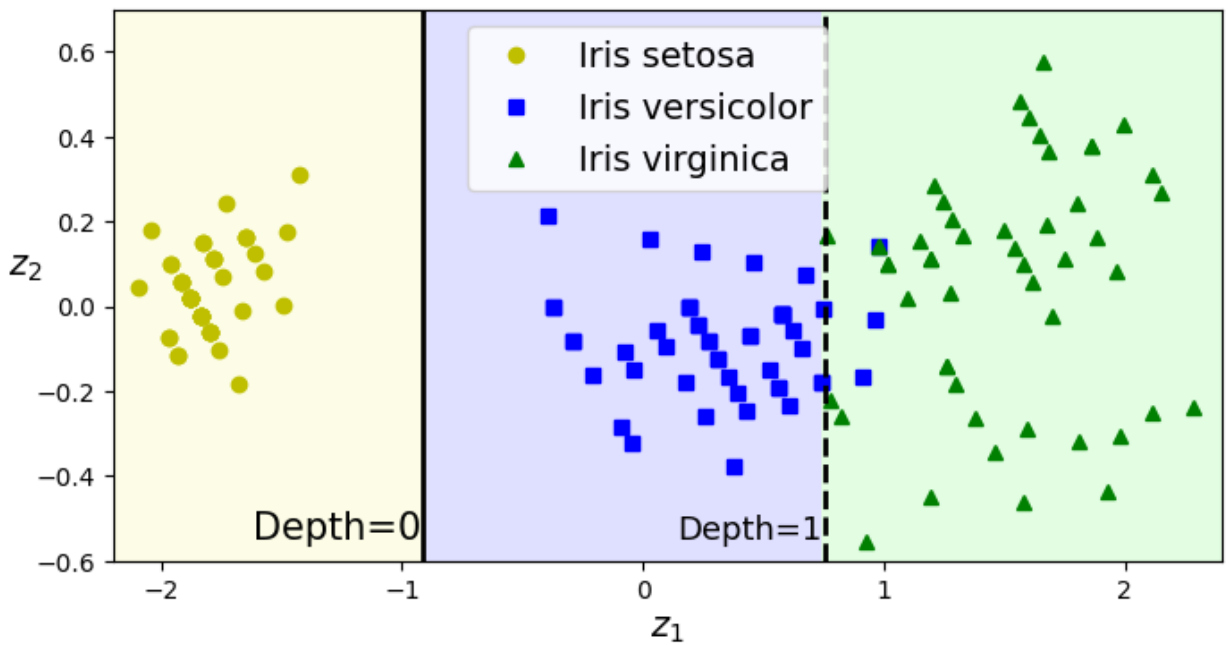
Los árboles CART de scikit-learn realizan divisiones del tipo $x_j \leq t$.

- Las fronteras son paralelas a los ejes de las características.
- Rotar los datos puede cambiar mucho la forma del árbol.
- En ciertos problemas, una transformación como PCA puede ayudar a obtener fronteras más simples.
- La escala de los atributos no es el problema principal: importa su orientación respecto a los ejes.

Ejemplo Iris:

```
from sklearn.decomposition import PCA
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

pca_pipeline = make_pipeline(StandardScaler(), PCA())
X_iris_rotated = pca_pipeline.fit_transform(X_iris)
tree_clf_pca = DecisionTreeClassifier(max_depth=2, random_state=42)
tree_clf_pca.fit(X_iris_rotated, y_iris)
```



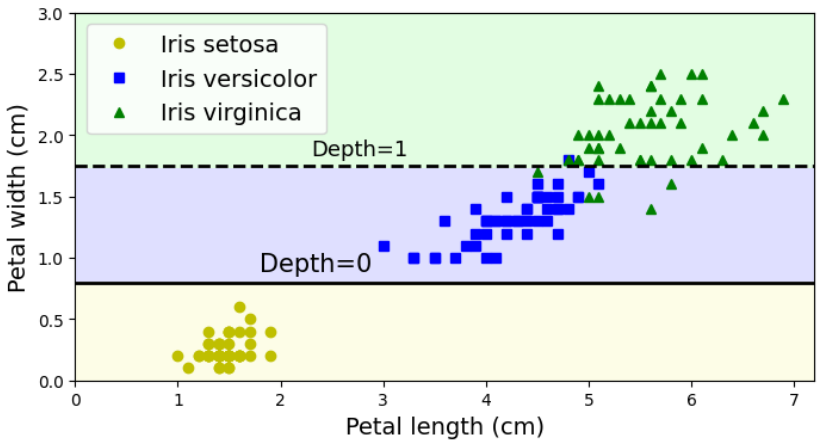
Alta varianza de los árboles

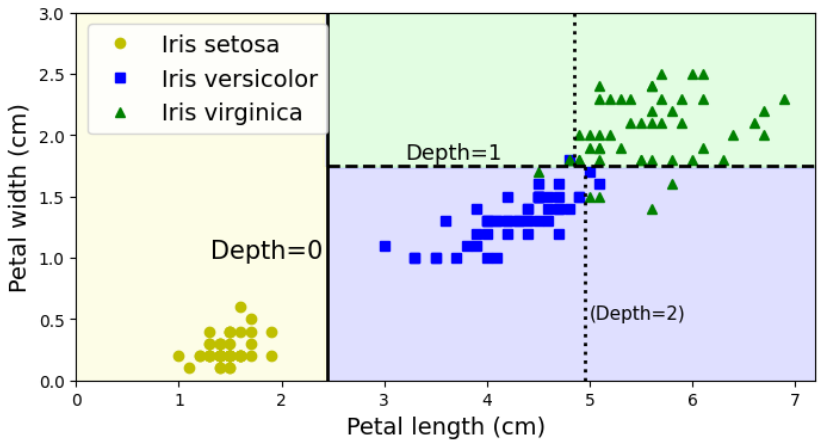
- Pequeños cambios en los datos pueden modificar considerablemente el árbol.
- Cambios en hiperparámetros también pueden producir estructuras distintas.
- Fijar `random_state` facilita reproducir los resultados, pero no elimina la sensibilidad del modelo.
- Combinar muchos árboles permite reducir la varianza: idea central de los **bosques aleatorios**.

Conclusión

Los árboles son interpretables y flexibles, pero suelen requerir regularización y una evaluación cuidadosa de generalización.

Ejemplo Sensibilidad:





Resumen: clasificación frente a regresión

	Clasificación	**Regresión**
Predicción	Clase mayoritaria	Media de los objetivos
Criterio	Gini o entropía	Error cuadrático medio
Salida	Clase o probabilidades	Valor numérico
Regularización	Profundidad, hojas, poda	Profundidad, hojas, poda

Preguntas de cierre

1. ¿Por qué no es necesario normalizar las características?
2. ¿Qué significa e implica que CART sea voraz?
3. ¿Qué hiperparámetro modificar ante sobreajuste?

Ensamble de árboles

próxima clase...

¡Gracias!